

Problem-Sized Lean Worlds

An authority-separated architecture from AI search to public mathematical claims

WILL COOK*

September 2026

ABSTRACT

This paper describes an implemented architecture for AI-assisted mathematics under proof abundance. It separates reasoning scope, permission to change shared state, validation capacity, evidence class, public-claim authority, and reviewer attention. Each transition requires evidence appropriate to the next stage. Recorded failures can inform later search and validation; their evidential value depends on what the attempt established.

Work is organised around *problem-sized mathematical worlds*: formal declarations, experiments, source literature, failed mechanisms, open obligations, and a reviewed public boundary. Formal dependencies, authored mathematical meaning, and public claims have separate graphs. In the private workbench, a compiled comprehension packet gives an agent the endpoint and claim ceiling before proposing a route. It federates corpora by content digest, distinguishes proved results from open producers and closed routes, and reports omissions and exceeded context budgets. The contribution is the connection between research state, scoped changes, formal checks, and public explanation, with their evidential roles kept distinct. The related-work comparison claims no priority.

Computation can reject a route or supply finite evidence. Lean checks a proof of the formal statement in the source; mathematical review must still establish that the statement captures the intended problem and that the paper describes it accurately. Comparator checks selected propositions and their axiom boundary. Local review selection prepares result families for external registration and review. These checks leave novelty, importance, acceptance, and open-problem status to separate assessments.

The public repository supplies Lean source, a claim record, papers, navigation, and release checks that can be inspected from a fresh clone. A worked example follows a finite certificate for Erdős Problem 249 through the checks that prevent an unbounded public claim. The system is a working prototype; multi-user use has not been validated. As of 31 August 2026, the author had recorded neither a completed external cold-clone use nor an accepted external contribution. All eight problems remain open.

Main contribution and claim ceiling

Contribution. The prototype keeps six authorities separate and makes every transition from conjecture to public wording carry its evidence and its limit. Problem-sized worlds join formal dependencies, mathematical interpretation, failed routes, and reviewed claims without treating any one graph as authority for the others. **Demonstration.** The mechanism is exercised on a self-contained public Lean corpus and an audited finite certificate for Erdős Problem 249. **Ceiling.** No open problem is claimed solved, and external use remains untested.

**Authorship and AI use.* Will Cook built and directed the research infrastructure and reviewed the claims when he could. AI agents did most of the research and drafting. Cook did not independently verify every claim.

1 The problem: many kinds of evidence

Suppose an AI system proposes a proof of a mathematical statement. Several different questions immediately arise.

1. Did the system understand the intended problem?
2. Did it explore the important alternatives and notice counterexamples?
3. Does the proposed formal statement say what the informal statement says?
4. Does Lean accept a proof of that exact formal statement?
5. Does the public explanation stay within the proved scope?
6. Has anyone outside the producing system reviewed, accepted, or absorbed the result?

These questions require different tests. A numerical experiment can expose a false conjecture; Lean checks a formal theorem; a release checker preserves a recorded decision about wording. None settles the other questions, and external attention alone does not validate a proof. The architecture records how work passes between these stages.

The [public repository studied here](#) contains Lean source, papers, and release machinery around eight open Erdős problems. At this revision all eight problems remain open. The repository contains substantial intermediate theorems, exact reformulations, conditional reductions, finite certificates, and no-go results. The surrounding private workbench is broader: it supports research, agent coordination, computational experiments, formalisation, exposition, and controlled public projection. Private machinery explains how the work was produced; it supplies no hidden proof authority to the public clone.

The central engineering problem is preserving the meaning of a result when it crosses these representations. A theorem name locates a declaration; it does not specify which informal sentence that declaration supports. Conversely, a paper can explain a sound argument whose steps have not all been formalised. The implementation therefore records the correspondence explicitly and checks that subsequent edits preserve it. Section 7 follows one such correspondence from a finite certificate to its public limitation. The [public architecture guide](#) supplies the clone-local map; the private mechanisms below describe the production design and are not additional services that a reader must install.

Contribution and scope. The proposed contribution is an executable *claim-transition architecture*, assembled from established components including agents, queues, file locks, theorem graphs, Lean checking, pull requests, and credit records. Reasoning, mutation, validation, interpretation, publication, and review have separate resource limits and permissions. Explicit fan-in gates bring their outputs together. Proofs, counterexamples, no-gos, and unresolved obligations pass through the same machinery with their evidence classes preserved. A local failure can change a later agent’s route, context, lease, experiment, or validator without changing a mathematical statement’s truth status.

Four terms distinguish implementation from proposed use. *Implemented* means that source and an executable check or receipt exist. *Projection* is a generated view over more authoritative records. *Inactive* means that implemented machinery was not running at the reported snapshot. *Hypothesis* denotes a proposed experiment. The router, work leases, corpus maps, Lean gates, Comparator, review-selection records, and release checks are implemented; dashboards and graph views are projections. The resident maintenance daemon was inactive at one recorded snapshot. Mass frontier-model mining and training on the no-go graph remain hypotheses.

1.1 Two architectural claims

Problem neighbourhoods. An agent works in a bounded neighbourhood of a problem-sized world. The neighbourhood starts with the exact endpoint and current claim ceiling, then selects established premises, nearby declarations, open producers, consumers, alternative coordinates, executable experiments, scoped falsifiers and no-gos, source literature, and public boundaries. Each edge records its basis and evidential limits; the packet lists omissions and expansion routes. File proximity, lexical similarity, and model interpretation help navigation but supply no proof.

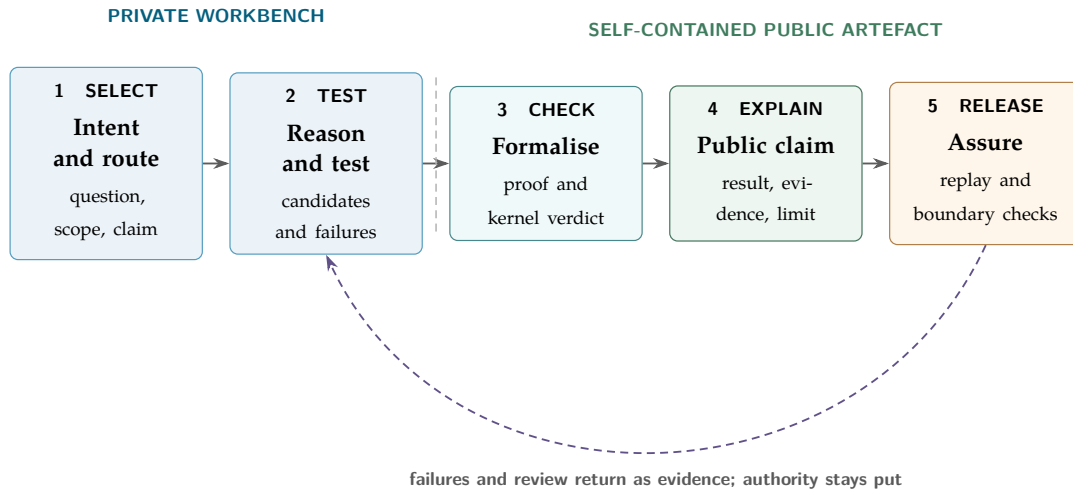


Figure 1: The end-to-end architecture. The dashed vertical line marks the release boundary. Everything to its right remains usable without the private workbench. The labels describe the production deployment; a contributor can also select and test new mathematics using the public clone.

Separate permissions and evidence. The architecture treats six resources as non-fungible. Reasoning scope does not confer a write lease, and a write still requires validation. A Lean receipt establishes formal acceptance, leaving semantic review, novelty, and community acceptance to their respective gates. Typed artefacts record these transitions.

Resource	Governing object	What possession cannot imply
Reasoning scope	Task-conditioned context and trace	Mutation permission or mathematical truth
Mutation permission	Exact path/work lease and change	Formal acceptance or public promotion
Validator capacity	Single-flight slot and build receipt	Theorem failure when a request is deferred
Evidence class	Experiment, counterexample, theorem, or authored relation	Automatic promotion into another class
Public-claim permission	Reviewed claim record and release relationship	Novelty, independent review, or acceptance
Reviewer attention	Ranked packet and recorded human outcome	Proof authority or canonical status

Table 1: The six non-fungible resources of the claim-transition architecture.

Mathematical and operational evidence update different graphs. The mathematical graph records theorems, counterexamples, no-gos, and bounded experiments with their evidence classes. The control graph records route misses, stale views, repeated workarounds, validation failures, and resource conflicts. After a generalisation guard, these may justify changes to a router, skill, check, or standard. Reviewed claims and publication artefacts connect the graphs without allowing either to determine the other's conclusions.

2 The whole lifecycle in one picture

Figure 1 shows the main path and its feedback loop. Failed proofs, reviewer objections, and public drift can inform earlier stages by identifying errors or better methods. This feedback does not alter what Lean checked or make an unproved statement true.

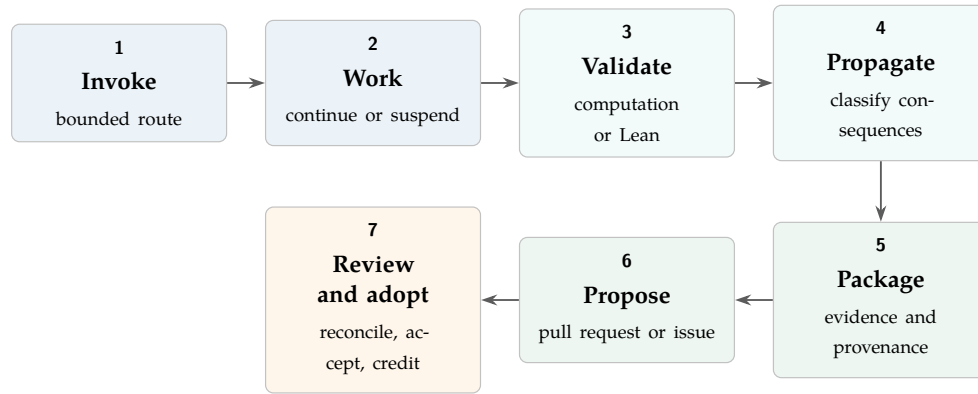


Figure 2: The clone-local job lifecycle. Closure means that the present delta has evidence and dispositions; it does not mean that the open problem is solved.

The lifecycle has four recurring operations. First, *selection*: choose a bounded object rather than treating the whole repository as one prompt. Second, *execution*: run an experiment, edit a proof, or write an explanation. Third, *validation*: ask the authority appropriate to that object. Fourth, *binding*: record the result, limitation, and source so a later agent does not have to rediscover them. The final feedback step turns a local success or failure into a reusable route, check, or warning when it genuinely generalises.

The public [proof-cockpit card](#) gives an agent this separation. It reads checkout and toolchain identity, corpus scale, registered open propositions, problem obligations, and workbench sessions from public files. It imports no private state and adds no evidence layer to Figure 1. Its `--check` mode runs the claim-registry, cold-clone, and projection-freshness checks; neither mode invokes Lean or decides whether prose follows from a theorem.

Once a person has compared a formal theorem with its public wording, a program can preserve the resulting decision about names, files, wording, and limits. It cannot decide whether the decision was correct. The workflow does not technically force a second independent mathematician: one maintainer can edit the Lean statement, record, and prose together so that every comparison agrees with the same mistake.

2.1 Skill-addressable job lifecycles

The public clone gives each recurring job one entry skill. A skill owns the job’s starting state, permitted mutations, required evidence, stopping or re-entry condition, and next owner; it does not acquire the authority of the objects it coordinates. Figure 2 records the ordinary research path.

The stable roles are orientation, bounded discovery, Lean validation, consequence propagation, return packaging, proposal, and maintainer adoption. Problem admission and manuscript revision have separate lifecycles. The [skill catalogue](#) lists the current entry names and links to definitions containing the executable details.

3 The private workbench

3.1 Disk is shared memory

Durable project state lives in source files, structured records, append-only events, generated views, validation receipts, and authored explanations. Conversations propose changes; files and their governing checkers determine whether those changes exist. To resume work, the next agent reads a bounded current-state packet and the relevant sources, so it need not rely on a summary of an earlier conversation.

File classes have distinct roles. Raw operator language preserves the request, and work records identify the claimed work and its owner. Source files contain implementations and proofs; receipts record what ran. Authored papers explain the work, while generated indexes provide navigation back to the governing records.

At entry, a deterministic router selects a small context packet containing candidate objects, authority boundaries, relevant standards, and permitted next actions. The packet guides source inspection; its summary has no proof authority.

3.2 Type A and Type B

Type A means a coding agent working in the repository, such as Claude Code, Codex, Cursor, Antigravity, or OpenCode. *Type B* means a chat AI used through a web interface: the operator gives it a selected research packet and brings its response back to the repository. This is a distinction in substrate access, not model quality: it does not rank either AI's intelligence.

Actor	What it can do	Governing limit
Type A	Inspect live project state; use repository tools; edit claimed files; run tests; bind receipts and status.	It may change only the substrate and paths its task authorises, and its claims remain limited by the relevant validator or human review.
Type B	Read a selected packet in web chat; return research, critique, or a candidate for the operator to carry back.	It has no direct access to these project files or tools. A Type A agent or the operator must check and apply useful output.

Table 2: Type A/B is an authority distinction. Either type may be weak or strong; a delegated tool-using agent is still Type A if it has live substrate access.

Type B output can inform research without being treated as an observation of private state. Type A changes remain attached to their paths, tests, and receipts. Delegation alone does not make a worker Type B: its access to the repository determines the role.

3.3 Concurrency without shared-state confusion

Several agents may work at once, but concurrency is admitted only where the write scopes can be separated. A work item identifies the objective and expected evidence. Before mutation, an agent claims exact paths for a bounded lease. Independent claims may proceed concurrently; overlapping claims must be coordinated or deferred. Fan-out is followed by a fan-in barrier where the results are compared, validated, and integrated.

Append-only ledgers and immutable receipts record coordination state, and generated status pages present views over them. They distinguish an active agent from an existing change and an accepted change. Bounded job counts and focused Lean builds limit resource use and prevent concurrent builds from corrupting one another's evidence.

3.4 The control plane is executable

The router exposes typed entries for skills, standards, paper modules, live work, and source artefacts. It selects a task-conditioned packet before the agent opens large sources. A cold agent can proceed from a one-line flag to a short card, a working context, and finally the governing file. Each layer identifies its source, expansion route, and authority, making the selection of context inspectable.

The work ledger records completed, reopened, and superseded work as append-only lifecycle events. Present permission comes from short leases over exact paths or work objects. Directory claims collide with their children; expired leases lose authority unconditionally. A crashed holder leaves an expiry and dirty-handoff record. Generated cohort views report activity, collisions, and unknown scope, but cannot grant permission to write.

A resident-runtime layer called metabolism turns repository events into bounded maintenance jobs. One daemon owns a SQLite store in write-ahead-log mode containing events, jobs, runs, provider budgets, heartbeats, and temporal blackboard claims. Hooks, filesystem scans, provider interruptions,

and explicit commands feed the store. Stable digests deduplicate events; an allowlist limits execution; cooldowns regulate provider use. Expired owners are recoverable, and a single-resident guard prevents competing schedulers. Status pages report the store's state without authorising repairs. At one snapshot used for this revision, the status surface reported the daemon as not running while queued jobs remained visible. That snapshot demonstrates persistent state in the implemented machinery, not an active maintenance service.

The operating loop observes the current state, classifies the task, routes it to an owner, claims the work, acts, validates, records the result, propagates reusable lessons, and observes again. Continuous work repeats this loop with explicit checkpoints and a new receipt for each advance.

3.5 What continuous mathematical work looks like

A continuous goal records a fixed research question, a changing frontier, and an explicit claim ceiling. Each run resumes from the latest accepted state: sources read, routes ruled out, computations interpreted, formal obligations still open, and the precise statement that remains unproved. The agent selects the highest-value available transition, such as closing a lemma, producing a counterexample, sharpening a reduction, repairing a source claim, or isolating a smaller obstruction.

The public continuation interface makes part of this design inspectable. [The continuation program](#) records a bounded session at a named starting commit, checks a returned result against that session, and packages the return for inspection. It composes existing query, workbench, and return-validation tools. It does not execute commands merely because they appear in a contributor's return. Its optional replay checks stored session probes; it is not a repository build or release check. Persistence here means that the next contributor can recover the question, starting source, attempted work, and evidence without recovering the original conversation.

A compressed trace can omit paths or worker outcomes. Repository state, command results, diffs, formal builds, and commit objects must establish whether a change occurred. The workbench therefore records completeness explicitly. Private activity traces document production; their event counts are not used here as public evidence of throughput or mathematical progress.

Search and validation are interleaved, with scoped failed routes retained and concurrent work permitted where tasks can be separated. A claimed advance requires the relevant artefact and receipt. If an unavailable validator, a human decision, or a new idea is needed, the run preserves its endpoint and a re-entry point. During long computations, an agent can advance independent proof, source, exposition, or adversarial work where it is safe to do so.

3.6 Coupled continuous goals: discovery and stewardship

Discovery and stewardship address different scales of the corpus. A problem-directed miner works near the proof frontier; assessing whether its new lemma is routine, weaker than an older theorem, or prominent enough to lead a paper requires a wider view. The *discovery goal* reads the current problem world, runs discriminating computations, attempts proofs, records falsifiers, and returns the smallest stable mathematical delta. The *stewardship goal* compares that delta with the current corpus, groups related declarations into result families, and reconciles downstream uses. The two roles retain separate responsibilities and authority.

The stewardship pass makes four decisions separately. *Authority* records what Lean, a computation, a paper argument, or an external source actually establishes. *Mathematical appraisal* asks about logical reach, mechanism depth, independence, sharpness, reuse, and the surviving open boundary. *Exposition placement* determines what leads the abstract, which theorem receives the longest explanation, and what remains subordinate. *Work allocation* determines which missing implication or consumer deserves the next unit of compute or expert attention. A result can be highly significant but mechanically unready, fully checked but mathematically routine, or ideal as a worked example without being the corpus's strongest theorem. No single scalar or status is allowed to decide all four questions.

The steward can detect that a purported advance restates the open problem, that several theorem names express one result, or that a paper buries a stronger theorem. It can also find weaker Comparator interfaces and Palomar packets that omit the hard mechanism. It may narrow, regroup, reorder, or defer these presentations. Such judgements do not supply a second proof authority: Lean validation, intended-meaning review, prior-art assessment, and community acceptance remain separate.



Figure 3: The coupled continuous-goal protocol. A landed mathematical delta is a reason to resume stewardship; a changed appraisal or consumer gap can change the next mining target. The protocol calls for a new pass only after a relevant change.

The clone-local [coupled-goals skill](#) specifies the division through [bounded discovery](#) and [consequence propagation](#), with source-pinned hand-offs between the jobs. The skill instructs an agent or controller; it supplies no resident scheduler. One agent can perform the jobs sequentially, or separate tasks, machines, models, or people can hold the roles.

A landed theorem, counterexample, authority change, paper correction, or external review outcome triggers a stewardship pass. The pass produces a source-pinned appraisal and a changed consumer or frontier disposition. This can direct discovery towards a stronger target, a missing hypothesis, a counterexample request, or a cheap discriminating computation. When neither the frontier nor a downstream consumer has changed, both goals yield.

Before either role begins, `explain-public-system` provides a separate cold-clone orientation job. An agent can read the public corpus and companion papers, adapt the explanation to a lay reader, mathematician, formaliser, compute contributor, reviewer, or infrastructure contributor, and point to the exact evidence behind its account. The human therefore need not learn the file layout before asking a useful question. Explanation remains a projection: it cannot promote its own summary into proof or acceptance.

4 The mathematical reasoning loop

Mathematical work starts by stating the problem in ordinary language with its known status and source. Candidate mechanisms are ranked by endpoint proximity, depth, independence from other routes, usefulness for future work, and overclaim risk. Theorem count and ease of formalisation do not determine that ranking. A decisive obstruction or close conditional reduction may deserve more attention than many routine lemmas.

4.1 Experiments are route selectors

Computation serves three useful roles. It can find a counterexample, measure a finite pattern, or test whether a proposed mechanism is plausible enough to formalise. Each experiment records its inputs, code, finite domain, output, and interpretation. The interpretation must state what the experiment cannot show.

A finite computation becomes formal evidence only through an explicit bridge. For example, Lean may evaluate a finite proposition with `decide` and check the resulting proof term. Even then, the conclusion remains finite. A pattern observed for many inputs does not become an “all inputs” theorem, and a successful numerical approximation does not become an exact equality. Failed experiments are kept when they prune a natural route; otherwise later agents pay to repeat the same mistake.

Negative evidence. A failed agent attempt records an uncompleted search path. A counterexample refutes a conjecture on its stated domain, while a Lean no-go theorem rules out a strategy class under explicit hypotheses. Counterexamples and no-gos can prevent repeated work and identify a missing ingredient for the endpoint. The corpus keeps problem-specific reasoning, proof gaps, coefficient-only and fixed-precision obstructions, and corrected statements alongside proved theorems. Each no-go retains its scope, so the failure of one mechanism does not rule out other proofs.

4.2 Status, formal acceptance, and mathematical judgement

The workflow distinguishes three senses of “verified”. Experiments guide route selection without supplying any of these judgements by themselves.

1. A *status oracle* says what a public registry or the literature currently reports about the problem.
2. A *formal oracle* says whether the pinned Lean environment accepts this exact statement and proof under the permitted axioms.
3. A *research-validity judgement* asks whether the formal statement matches the intended problem and whether the result matters in context.

The second can be mechanical. The first can become stale. The third remains a mathematical and scholarly judgement. A successful Lean build without statement reconciliation is therefore not the end of the reasoning process.

Each iteration begins with the existing Lean and semantic graph. Proof search and bounded computation probe gaps and patterns that may be too distributed for an unaided manual scan. A checking pass records the kernel result, reconciles formal and informal statements, and packages a theorem, counterexample, or diagnosed failure. Accepted nodes and edges inform the next search; whether they improve it remains an evaluation question. Reusable process lessons can change heuristics and context, without changing mathematical truth. For mechanisms drawn from the literature, annex records, locators, and public citations retain the original authorship and claim scope. Lean checks the new formal statement.

4.3 From local progress to reusable mathematics

A lemma found in one Erdős problem may yield a comparison principle, finite combinatorial device, analytic estimate, or Lean interface useful in a wider library. The system distinguishes *process up-propagation*, which improves how later agents work, from *mathematical canonicalisation*, which develops a local result into a reusable theorem.

Canonicalisation requires examining which hypotheses the proof uses, removing problem-specific coordinates, checking prior art and existing library interfaces, and stating a natural general theorem. The local result should be an explicit specialisation. Dropped hypotheses require counterexample tests, and the general proof needs its own exact check. At least one additional consumer or explanatory use should establish the value of the generalisation; broader notation alone is insufficient.

This yields three statuses that must not be collapsed: checked inside one problem world; proposed as reusable mathematics; and suitable for an external shared library. The present system can preserve the first, help construct and test the second, and prepare evidence for the third. It cannot grant the third status to itself. Mathlib is open to new contributors, but its current guide sets high standards for generality, integration, maintainability, style, documentation, and responsible disclosed use of AI [13]. A subject and Lean expert must decide whether a candidate belongs there, reshape and review it as needed, and take responsibility for any upstream discussion or pull request. The originating route should remain visible in provenance; the expert should receive explicit credit for the generalisation, library design, formalisation, review, and stewardship actually supplied. The [open-source strategy](#) gives the corresponding participation and credit protocol.

4.4 Problem-sized Lean worlds and bounded theorem neighbourhoods

The public corpus provides several maps. The [declaration atlas](#) locates statements. [Dependency queries](#) distinguish module imports from constant references extracted from elaborated Lean declarations; unavailable or stale extraction cannot establish that a dependency is absent. The [authored semantic layer](#)

explains mathematical roles such as specialisation, transport, and scoped obstruction. The [claim record](#) connects selected results to reviewed wording and exact open propositions. A dependency can guide premise selection without establishing an authored analogy, and an analogy can suggest a proof without supplying one.

The public Problem 1041 materials illustrate why these distinctions matter. The [problem index](#) identifies the integrated library and its separately governed research export. The export's [manifest](#) inventories its files; its [frontier account](#) records surviving mechanisms and refutations; and its [selected-results map](#) binds statements to source with their premises and falsifiers. An exported file, an integrated module, a semantic interpretation, and a reviewed public claim are different objects. The declaration and semantic inventories expose their respective coverage; generated certificate families make raw theorem counts a poor measure of mathematical value.

The problem cockpit first fixes the endpoint, status, claim ceiling, and canonical frontier. It distinguishes open producers from the target, so a promising route does not replace the problem. A bounded neighbourhood then selects the strongest relevant results, exact source coordinates, imports, consumers, alternative formulations, experiments, counterexamples, no-gos, and literature. Each item has an evidence class. An omission receipt records what was excluded and how to retrieve it. For an exact claimed path, a local connection card can present prerequisites, sibling mechanisms, consumers, falsifiers, and the validation target before broader retrieval.

4.5 Comprehension before the mathematics

The private workbench compiles this neighbourhood into a packet conditioned on the agent's query. The following federation and inference-workspace behaviour belongs to that workbench. In a public clone, the independently usable [corpus query tool](#) supplies local claim, declaration, dependency, and connection-card routes; it does not require the private compiler or its attached corpora.

Infrastructure questions have a separate entry path. The private workbench offers a small choice of architecture, source layout, local connections, validation, experiment evidence, and paper ownership; the agent selects an existing owner without scanning the proof corpus to discover its organisation. The public clone's [task-entry tool](#) routes architecture and navigation work to its [infrastructure maintenance workflow](#). In both settings, source topology locates work, module imports guide inspection, and a build plan selects validation. These routes help an agent choose what to inspect or run; passing their structural checks does not establish a mathematical claim.

Each corpus retains its own authority. The compiler stores compact descriptors and content fingerprints for the private projection and attached public checkouts, without copying them into a central index. It opens an attached exhaustive atlas only after the identities reconcile and bounded retrieval fails to answer the query. It does not rebuild the expensive Lean microcosm. Every packet names the corpora consulted and their digests.

The packet leads with the endpoint. A problem cockpit fixes the target statement, its status, the claim ceiling, and the current claim frontier, labelling each frontier claim with an evidence class and a logical altitude that separates an equivalence from a reduction, a necessary condition, a finite exclusion, and a no-go. Beneath it a mechanism landscape keeps proved or kernel-checked results apart from open producers and from routes a counterexample has closed. Retrieval moves between global frontier structure, concept neighbourhoods, and premise or obligation ancestry, and exact imports, authored arguments, generated navigation, lexical references, and semantic inference remain visibly different edge classes.

An inference workspace organises this material in three planes. The exact commitment plane holds source coordinates and statements. The strategy plane branches over coherent premise groups, and the adversary plane records the falsifiers each branch must survive. A changed corpus fingerprint invalidates the commitments. Every packet states the limit of this planning workspace: only Lean, an exact counterexample, or an owner verifier receipt may change a claim's status.

The compiler reports a query as unanchored if it names no theorem, declaration, or claim. Its consumer must name one before treating a route as target-specific. A packet that exceeds its requested context budget exits non-zero, reports requested and estimated token counts, lists dropped planes, and gives commands to restore them. If the adversary plane was omitted, the packet records that omission and an expansion route; it does not report an absence of falsifiers.

A correctly identified and well-formed packet can still select a weak mechanism, omit the governing obstruction, or carry a mathematically mistaken interpretation. The supported claim concerns its starting state: bounded context with source digests, evidence classes, and declared omissions. The packet has no proof authority. This paper does not measure whether it helps agents produce better mathematics.

4.6 Consequence propagation

When a declaration lands, the direction reverses. A consequence mapper begins at the exact changed object and enumerates reverse imports, claim records, validators, experiments, papers, and open obligations that may now be stale. A semantic second pass must choose: update now, verify unchanged, defer with a reason, or mark outside scope. Empty lexical search is not evidence of no consequence, and no projection may bulk-strengthen a family of claims. A formal evidence cell carries the result, its explicit non-claim, evidence class, source and receipt links, and next unresolved obligation through this fan-out. The public [consequence-propagation skill](#) specifies the same discipline without depending on the private workbench. For work returned from an older clone, it runs once against the contributor's recorded starting commit and again after the reviewed change has been reconciled with current main. The original delta and any conflict-resolution delta remain separate evidence and receive separate attribution.

This same architecture distinguishes mathematical propagation from process propagation. A theorem, counterexample, or no-go changes the mathematical graph only through its own verifier. A reusable failure in navigation, scheduling, experimentation, or validation enters a guarded packet containing the local case, failure class, evidence, sibling scan, proposed owner, overgeneralisation guard, validation, and stop condition. If accepted, it may change a route, skill, check, or standard for later agents. The mechanism is implemented and has worked examples, but there is not yet a complete measure of what fraction of useful failures propagate. The supported claim is that failures *can* alter the durable control plane without altering mathematical truth.

5 The public Lean repository

The public checkout contains the source needed to inspect its claims and replay its checks, with no calls to the private workbench or unpublished private lemmas. Builds also require the public toolchain and dependency versions recorded by [lean-toolchain](#) and [lake-manifest.json](#), which a fresh checkout may need to download. Self-contained therefore describes the source and its authority, rather than an offline installation. The private workflow remains production provenance.

The repository has two Lean roots. The reviewed root contains the established 249/257 publication lane. The problem-owned expansion root contains work on six additional Erdős problems and unpromoted lanes for 249 and 257. Both roots are checked by the same Lean kernel, but kernel acceptance does not automatically promote a declaration into a reviewed public claim. Promotion is a separate change to the claim record and exposition.

The public sources divide responsibility as follows:

Surface	Authority	It does not establish
Lean source and pinned toolchain	Exact formal statements and proofs accepted by the kernel.	Intended meaning, novelty, significance, or faithful prose.
Reviewed claim record	Approved wording, status, evidence, bounded domain, and adjacent open statement for selected claims.	Correctness or completeness of the human review.
Papers and guides	Human explanation and reading order within the recorded claim ceiling.	New formal authority.
Generated maps and query tools	Bounded navigation across declarations, problems, graphs, papers, and claims.	Proof or permission to strengthen a claim.
Release checks and continuous integration	That configured identities, relationships, generated files, licences, and negative tests pass on a named revision.	Understanding of unrestricted prose or independent mathematical approval.

Table 3: The public authority split.

The companion public Plectis repository has a different role. It publishes runnable, bounded mechanism slices and receipts from the wider system. The Lean repository publishes a mathematical corpus. Neither public repository inherits authority from the other, and neither is a public mirror of the private root.

6 Comparator, Palomar, and publication

6.1 Comparator: selected statement interfaces

Comparator requires selected propositions to be stated again in a challenge module without their proofs. A solution module must provide terms of those exact types. The [configuration](#) names the permitted axioms, modules, and runtime receipt; the [verification guide](#) explains their roles. A named altered statement must fail, testing that the harness is not vacuous.

The check establishes that the proof-bearing corpus implements the separately stated Lean interface under the specified axiom budget. It leaves the translation from informal mathematics, novelty, importance, and independent human review unresolved. The appropriate description is “Comparator-checked”; the receipt alone does not establish independent verification.

6.2 Review selection and the Palomar registry

Comparator inventories evidence. Local review selection groups declarations into result families and ranks them by mathematical signal to choose a small portfolio for review. Each packet keeps the hard mechanism beside the surviving boundary. The [qualification record](#) records whether the packet satisfies its structural requirements. The [local selection record](#) uses Palomar’s name because selected families may be submitted to its external registry of Lean-verified mathematics. Palomar documents mechanical proof checks and automated editorial filtering, without human peer review or significance ranking. Local qualification, registry inclusion, and human mathematical review are separate events; none alone establishes broad community acceptance or significance.

The local ranking asks which result changes the mathematical picture, which conditional route approaches an endpoint, which obstruction saves repeated work, and which explanation would help an

expert assess the claim. These criteria direct attention without altering proof status or rewarding theorem count and generated volume by themselves.

6.3 Publication and propagation

After formal proof and statement reconciliation, a result may enter an authored paper, a claim record, a Comparator packet, and a local review unit. Each records a different decision with a different ceiling. The [publication contract](#) inventories the manuscripts and rendered files. The [release program](#) and [continuous-integration workflow](#) check the recorded relationships. Release policy requires both the Lean build and the public-surface checks; the Python release program does not itself elaborate Lean.

Failures and review feed back into the appropriate owner. A proof failure can change a formalisation heuristic, a counterexample can close a route, a reviewer objection can narrow a claim, and release drift can prompt a new check. The resulting change belongs in a theorem, experiment record, skill, standard, route, or claim boundary. It must not rewrite raw intent, generated views, or mathematical status by implication.

6.4 From formal checking to mathematical use

Tao argues that AI mathematics should be judged along a chain: proof generation, verification, exposition, publication and community digestion, then eventual canonicalisation [12]. This architecture is a concrete attempt to build for that whole chain. The private reasoning and experiment loops support generation. Lean checks formal proofs, while Comparator protects selected exact interfaces. The problem papers, reasoning surfaces, graphs, and claim records support exposition. Local review selection triages scarce expert attention, and the contribution and release paths support attributable publication. Digestion, acceptance, and canonicalisation remain achievements of the mathematical community, never local status fields.

Tao warns that polished AI exposition can erase the friction that reveals where the mathematical difficulty lies. The reasoning surfaces and no-go graph retain false starts, corrected claims, missing producers, and formal obstructions alongside the successful theorems. They help a reader understand why the surviving boundary is difficult and where a new idea is needed.

Paper authoring itself participates in this loop. As the mathematical and control graphs change, agents assemble and revise problem papers, architecture papers, reviewer cards, and claim records from the source-current evidence. We call this AI-assisted preparation *pre-digestion*. Agents organise arguments, expand intermediate steps, produce examples and diagrams, and link statements to their sources. A missing explanation, unstable term, hidden quantifier change, or unregistered limitation found during that work becomes a defect to investigate in the theorem, claim record, or software.

A digestive-system analogy distinguishes breaking food down from absorbing its nutrients. Similarly, preparing an argument in an accessible form does not establish that a mathematician has absorbed its ideas. Mathematical digestion requires people to reconstruct the reasoning, examine where the assumptions matter, explain the difficult step, and connect the result to mathematics they can use. AI can assist with this work; generated exposition cannot certify that it has happened. This is our analogy for Tao's distinction between verified proofs, readable exposition, and results understood and accepted by mathematicians [12, Sec. 6, pp. 6–8; Sec. 8, p. 11]. Formal checking, demonstrated human understanding, and community acceptance remain separate forms of evidence.

This manuscript is a reflexive example: it was rewritten by the system while agents inspected the mechanisms it describes, then compiled and checked as a public artefact. Self-authorship records production and supplies no independent validation; a clear account does not constitute an independent review.

Tool use is disclosed and models are not listed as authors. Human contributors receive differentiated credit, and references and source paths remain inspectable. Automatic checks filter work without replacing peer review; theorem counts do not determine mathematical value. These choices make the repository an implementation experiment in preparing a small project for proof abundance.

7 One complete boundary: finite is not unbounded

Erdős Problem 249 asks whether

$$S = \sum_{n \geq 1} \frac{\varphi(n)}{2^n}$$

is irrational, where $\varphi(n)$ is Euler’s totient function [2]. We use this example because its finite and unbounded statements can be followed through every publication layer; it is not a ranking of the corpus’s strongest mathematics.

The development reduces irrationality to exact non-integrality certificates. For $t \in \mathbb{N}$, let $H_t = \text{lcm}(1, \dots, t)$, with $H_0 = 1$, and write

$$\text{Cert}(t) \iff \exists L \in \mathbb{N}, \text{certifiedKill}(H_t, H_t, L).$$

The [certificate predicate](#) uses a finite integer truncation of the difference of two totient tails. It requires the truncation’s residue modulo 2^L to avoid both ends of the residue interval by an explicit tail-error bound. Thus a certificate can be checked using finite integer arithmetic, although its consequence concerns an infinite series.

Lean checks the finite theorem

$$\forall t \leq 82, \text{Cert}(t).$$

This is the [finite diagonal-certificate theorem](#). The endpoint needs an unbounded supply:

$$\forall T, \exists t > T, \text{Cert}(t).$$

The [diagonal-supply equivalence](#) proves that this unbounded statement is equivalent to irrationality. The source uses $\forall t_0, \exists t \geq t_0$; over natural numbers this is equivalent to the strict-cutoff form displayed here. It does not prove the missing implication from the finite range to the unbounded statement. No matter how large a fixed checked bound is, a larger cutoff exists.

The example connects the layers directly. Computation constructs the finite data that Lean checks through the certificate predicate and finite theorem. The formal graph records dependencies and the semantic graph describes the result’s role. The claim record fixes the finite range and the open unbounded requirement, whose quantifier difference the paper explains. Comparator checks selected exact interfaces; local review selection may rank the family; the release checker requires the limitation to remain visible.

A historical README edit exposed a gap in the last check. It replaced a clause saying that the finite cases did *not* supply the open requirement with one saying that they completed it. Lean was unchanged. The release checker passed because the prose relationship was unregistered; after it was added to the claim record, the checker rejected a deliberately false copy. This demonstrates one failure and repair, without measuring detection rate, coverage of claim-bearing prose, or mathematical review quality.

Nine of the ten edits were rejected. One escaped because the relationship had not been registered. The original run logs were not retained. The edits were authored by the checker’s author. Only the escaped edit was reconstructed after the repair. The other nine edits were not rerun against the extended checklist. The finite theorem makes no $t = 83$ or cofinal claim. The [historical evidence record](#) documents this coverage boundary. The [reconstruction manifest](#) and [mutation harness](#) make the registered edits inspectable but do not replace the missing original logs. The post-repair witness accepts the current README and rejects a test copy containing the false clause.

Other lanes illustrate different claim boundaries. Problem 257 has a theorem for full-support representations; the stronger arbitrary-support statement remains open. Problem 68 has an exact carry-based equivalence without a theorem producing the required carries. Problem 251 gives an exact series reformulation without an irrationality proof, and Problem 243 gives a conditional recovery theorem whose premises remain in the public claim. The 1041 lane also records a counterexample and corrected statement. In each case, publication preserves the quantifiers, premises, and nearest stronger open statement.

8 Inspection routes

The following routes lead from common inspection questions to the relevant public sources.

Question	Start here
How does the public repository fit together?	docs/ARCHITECTURE.md
What may the project say, and what remains open?	docs/claims.json and docs/methodology.json
Where is the checked mathematics?	lean/Erdos249257.lean and lean/ErdosProblems.lean
How can I navigate without reading the whole corpus?	docs/ORIENTATION.md and scripts/query_corpus.py
What exactly does Comparator check?	docs/EXTERNAL_VERIFICATION.md and verification/comparator.json
What does the local review selection qualify?	PALOMAR_QUALIFICATION.md and docs/PALOMAR_RESULT_SHOWCASE.json
Which papers exist and what question does each answer?	docs/papers/README.md
Which checks gate a release?	scripts/check_release.py and .github/workflows/lean.yml
How can work return with public credit?	CONTRIBUTING.md and research-commons/CONTRIBUTIONS.md

Table 4: Compact public inspection routes. These are entry points, not a new authority layer.

9 What can be trusted

The architecture supports a chain of narrow conclusions:

1. a recorded experiment ran on its stated finite inputs;
2. a pinned Lean kernel accepted its stated formal source;
3. a maintainer approved a selected interpretation and public boundary;
4. Comparator matched selected proof-bearing declarations to separately stated interfaces and an axiom budget;
5. the local review-selection checks validated the structure of a selected review packet;
6. the release program found every configured public relationship intact on the named revision.

Together, the receipts trace a result through the system. They do not establish that its formalisation is uniquely right, its review independent, its contribution novel or significant, or its registered boundary complete. They also do not establish a solution to an open Erdős problem. A coordinated mistake in source, record, and prose can satisfy every structural comparison. Unregistered prose can overclaim, and literature status can change.

Multiple sources, explicit boundaries, and negative tests near high-risk claims help expose these errors. Mathematical judgement remains necessary, and the system does not require a second independent mathematician. Assurance cases have a similar limit: their notation can record a claimed support relationship without proving that the evidence is true or sufficient [11].

A capability described in this paper may have any of four kinds of support:

Kind of support	Example	What may be said
Executable check	A session or return validated against required fields and source identity; a Comparator interface match	The program checks the stated conditions on the stated inputs
Agent workflow instruction	The discovery and stewardship responsibilities in a public skill	The workflow directs the agent; it does not show that every agent complied
Mathematical judgement	Whether a result expresses the intended mathematics or improves on prior work	A named assessment is required; agreement between source, record, and prose is not one
External outcome	Another researcher reproduces, adopts, corrects, or builds on the work	A recorded external event is required; the architecture implies none

10 Scaling from one clone to a search network

10.1 Clone, attach compute, and mine bounded results

A fresh public clone fixes a Lean toolchain, exposes supported roots, records reviewed claims and open boundaries, and supplies query and release programs. An external researcher can attach an agent runner to these interfaces, assign disjoint theorem or experiment lanes, and submit candidate changes through the proof and publication gates. Added compute can increase the number and diversity of attempts without changing what a passing receipt means.

In the proposed “result mining” mode, workers select bounded frontier objects and return proof attempts, counterexamples, computations, or literature findings as exact artefacts. Fan-in checks Lean, reconciles statements, records negative evidence, and ranks the strongest surviving result families. Each result packet carries provenance, scope, formal status, mathematical role, and a nearest stronger open statement. Local selection allocates review attention to these packets, while Comparator protects selected exact interfaces.

At scale, stewardship would handle each stable delta as it arrives. It compares the delta with the candidate universe, updates the paper hierarchy and assurance interfaces when warranted, and returns a ranked frontier to the workers. Several miners can share one stewardship goal, and one steward can cover several problems, provided every write and mathematical object retains one owner. A large run may produce no new mathematical family; a short negative result may deserve more prominence than many formal lemmas.

The controller admits work according to path conflicts, memory, CPU, disk pressure, and the expected value of the lane. Many agents can in principle search disjoint mathematical neighbourhoods, while practical limits differ by activity. Reading and cheap computation can run widely; changes require path leases, and heavy Lean validation has narrower capacity.

Public Lean validation uses [single-flight request coordination](#): equivalent requests share a detached owner and may reuse its completed receipt. The request key conservatively includes all visible Lean source content, build authorities, toolchain and dependency-lock identities, and the normalised command, which includes the targets. It is independent of the checkout path; unrelated paper edits do not change the Lean request key. This is content-based reuse, not a claim that arbitrary equivalent commands or proofs are recognised.

Distinct requests still compete for the same host-wide Mathlib resource. The worker attempts a non-blocking host lock. If another Lean validation owns it, the request ends with a typed resource deferral, exit 75, and must be submitted again after that owner finishes. The transient state named `queued` does not promise automatic eventual execution. The ordinary [focused-build wrapper](#) submits and waits for its result; the explicit submit interface returns a receipt that can be collected later. This lets a controller advance independent work while an admitted build runs. A deferral reports resource availability, not a failed mathematical theorem.

Scaling therefore has four limits: logical search width, safe concurrent mutation, validation throughput, and expert review. Path leases govern mutation, admission controls validation, and result ranking directs reviewer attention. Fan-in integrates the outputs once the required receipts exist.

The return path is already public. A person can fork or clone the repository, work from a named public commit, and open a pull request or research-progress issue. A structured return can preserve the contributor, collaborators, tool operator, disclosed model systems, starting commit, evidence, affected result, and surviving limitation as separate fields. Only an accepted receipt enters the generated contribution views. Acceptance, mathematical claim status, and release inclusion remain separate decisions, so a credited counterexample or failed route need not be mislabelled as a theorem. Later corrections append a new history rather than erasing the earlier contributor. Attribution and pull requests are standard practice; the architectural role here is to carry credit and provenance through fan-in without allowing either to strengthen the claim.

The named starting commit remains useful even when main advances. Git retains the common ancestor needed to inspect the contributor's original delta. A maintainer can replay that state, reconcile the reviewed substance with the current tree, and rerun current validation and consequence propagation. A material conflict resolution is a new integration contribution; it does not rewrite the origin of the earlier work.

The exact contributor-facing sequence is specified in the [companion contribution protocol](#); its credit model is separated in the [provenance and credit section](#). The navigation assumptions made here are tested by the [cold-clone case study](#).

The public clone supplies the mathematical corpus and its gates. The full private orchestration layer is not distributed as a turnkey service, and mass multi-provider mining remains a design target without a reported benchmark. Public interfaces allow a laboratory to use another scheduler or producer while retaining the evidence boundary.

No outside contributor had completed this path by 31 August 2026. The cross-paper links, clone-local skills, and pull-request route are therefore implemented prototype interfaces whose usability remains an external test.

10.2 Using the no-go graph

The proposed graph records endpoint problems, conjectures and reformulations, mechanisms, computations, formal lemmas, counterexamples, no-go theorems, reviewed claims, and unresolved obligations. Its edges distinguish implication, equivalence, dependency, refutation, obstruction of a strategy, weakening, generalisation, experimental support, formalisation, and publication. A no-go rules out a class of mechanisms while leaving nearby variants to be examined separately.

As this negative and positive graph grows, several new uses become testable. A model may navigate around already closed strategy families, identify a small cut of missing premises between the current corpus and an endpoint, or search for analogies between obstruction patterns in different problems. Training data can be formed from proof/counterexample/no-go triples or from contrastive pairs consisting of a tempting argument and the exact theorem explaining its failure. Graph-conditioned models might propose the next useful lemma from a frontier neighbourhood rather than from the theorem statement alone. These are hypotheses about future systems, not results established by this paper.

Negative structure also creates risks. An over-broad obstruction edge can hide a viable variant; repeated failed attempts can teach stylistic avoidance rather than mathematics; and model-generated semantic edges can disagree with the formal graph. Training snapshots therefore need immutable generations, source-level provenance, explicit edge authority, and held-out evaluation. Lean can verify formal nodes and some edges, but expert judgement remains necessary for semantic roles and for deciding whether the graph captures the important mathematical space.

10.3 An experimental agenda

Evaluation should compare agents with and without graph context on frontier selection, repeated dead ends, time to a useful obstruction, premise discovery, proof completion, and calibration of public claims. It should test transfer between unrelated mathematical domains as well as neighbouring Erdős problems. Independent teams should replay result packets and assess the scope of no-go edges. The question is

whether the accumulated graph improves the efficiency and originality of later reasoning, including on mathematics beyond the corpus already mapped.

11 Relation to other approaches

Theorem-proving agents. LeanDojo and Pantograph provide programmatic Lean environments and proof-state interaction [14, 15]. OpenProver combines a planner, parallel workers, independent verifiers, compact working memory, a larger repository, and Lean feedback [16]. Agent Hunt studies concurrent formalisation with locks, bounties, guarded ownership, and collaborative agents [17]. DreamProver learns a compact reusable lemma library through wake-sleep cycles [18]. These systems provide baselines for proof interaction, parallel search, and learned proof memory. The present architecture contributes no new proof-search algorithm; it addresses the subsequent transition from a found proof to a reviewed public claim.

Graphs and checked exposition. Proof blueprints connect informal proof plans to named Lean declarations. *leanblueprint* checks that author-supplied declaration names exist, while *LeanArchitect* infers formal dependencies and unfinished-proof status and exports synchronised blueprint material [3, 4]. The graph layers here have a related navigational role, but the public-claim record begins after a result has been selected and asks which wording and open boundary were reviewed.

Semantic-audit systems address a neighbouring problem. Lean Atlas narrows the declarations a person must inspect for chosen theorem statements, conditional on the semantic correctness of the returned set and trusted base [5]. EconCSLib uses models to translate and compare formal and informal statements, with saved human judgements for a subset [6]. The present architecture uses models throughout production but assigns none of them final semantic authority. Their output becomes evidence or a candidate until a source-specific gate accepts it.

Isabelle/DOF places formal and informal material in a typed checked document [7]; requirements traceability follows commitments across development and revision [8]. This repository leaves prose unrestricted and records selected public boundaries separately, so unregistered prose remains outside the checker.

Auditable scientific agents. HEP records hypotheses, evidence, belief updates, lineage, and resolution states in an append-only registry [19]. Symposium proposes immutable community publication histories with attributable artefacts, declared evidence, assumptions, and purpose-sensitive arguments [20]. EurekaAgent includes permissions, artefacts, budgets, and human supervision in its agent environment [21]. These precedents establish the role of persistent records and environment design. The comparison here concerns the separation of experimental evidence, kernel acceptance, reviewed interpretation, exact-statement assurance, editorial selection, and public release.

Mutation testing asks whether a test distinguishes a seeded fault from the original program [9, 10]. The deliberately false boundary in the worked example follows that idea. Because the examples were selected by the system’s author and were not run as a controlled external evaluation, they show that particular checks can fail; they do not measure overall adequacy.

Contribution and limits. The contribution combines existing components into an architecture joining a private workbench and a self-contained public Lean repository. Formal dependencies, semantic interpretation, reviewed claims, Comparator interfaces, review selection, and release relationships have separate records. Failure modes and formal no-gos pass through the publication process with positive theorems, while retaining the stronger endpoint that remains open. Failure logging and hypothesis refutation have precedents; the comparison here concerns their integration with formal obstruction results, claim ceilings, and publication assurance. Evidence comes from the design and a worked reconstruction in one evolving corpus. It provides neither a controlled performance comparison nor an independent audit or general reliability estimate.

Scaling beyond this corpus. The design is not tied to Erdős problems. A new domain needs stable result identities, source and status records, a formal checker where one exists, and an explicit public-claim boundary; it need not use the same mathematics or even Lean. Larger deployments should federate independently owned corpora rather than build one authority database, separate producer and reviewer teams,

benchmark each boundary independently, and preserve immutable result generations. Proof-search layers can adopt distributed task markets, persistent lemma learning, and richer human steering, while publication layers can add external replay, signed review receipts, and cross-project claim links. None of these extensions should allow learned process memory or a graph edge to change mathematical truth by itself.

12 Conclusion

This architecture organises AI-assisted research around bounded neighbourhoods in problem-sized mathematical worlds. At each transition from search to public wording, it records the evidence, the person or program authorised to assess it, and the stronger statement that remains open. Reasoning scope, mutation permission, validator capacity, evidence class, public-claim permission, and reviewer attention have separate gates.

The private workbench supports durable research and concurrent changes. Type A agents work on claimed paths under checks; Type B agents return bounded reasoning from web chat. Experiments help choose routes, and Lean checks formal statements. Graphs support navigation, Comparator checks selected interfaces, and local selection prepares results for review. The external Palomar registry checks and filters submissions. Source, claims, papers, and release receipts are published in a self-contained clone.

The worked example shows how these records preserve a finite theorem’s boundary through publication, including the repair of one escaped overclaim. It does not establish an automatic mathematician or a verifier for arbitrary publication claims. Intended meaning still requires mathematical judgement; independent review, community understanding, and acceptance require evidence beyond the producing system.

A Reproducibility

Public first contact. A fresh public clone can reproduce the control card and structural checks:

```
python3 scripts/proof_cockpit.py --format card
python3 scripts/proof_cockpit.py --check
```

The first reads committed public metadata; the second checks the claim registry, cold-clone contract, and orientation freshness. Neither checks a proof. Formal authority begins with the pinned Lean build named by the card.

Source versions and coverage. Repository source links in this revision resolve at public commit 9b654f4cce44; they identify inspectable source, not a new validation run. The [publication entry packet](#) separately identifies the formal-source checkpoint, release identity, and evidence owners. A reader working from another revision should query those identities again before comparing its output with this paper.

The [semantic query tool](#)’s coverage command distinguishes exact proposition evidence, authored family context, and structural-only linkage. Contextual membership does not mean that every member received statement-level mathematical review. Coverage counts belong to the generated receipt at its source digest; they do not measure review quality, mathematical value, or public-claim completeness.

The [paper inventory](#) records source and PDF cryptographic hashes and validation commands. The [worked-example evidence](#) and [reconstruction manifest](#) specify the deliberately false edits and their limitations. Comparator’s configuration and the local qualification and selection records are linked in Section 6. These artefacts identify what was checked. They do not interpret unrestricted prose or confer external acceptance.

The private system is described here at the architectural level because it is production provenance, not a dependency of the public result. Private paths, operator material, unreleased work, and private ledgers are neither required nor granted authority by this paper. The public checkout remains the inspection and replay boundary.

References

- [1] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, in *Automated Deduction—CADE 28*, Lecture Notes in Computer Science 12699, 2021, pp. 625–635, DOI.
- [2] P. Erdős and R. L. Graham, *Old and New Problems and Results in Combinatorial Number Theory*, Monographies de L’Enseignement Mathématique 28, 1980, p. 61.
- [3] P. Massot, *leanblueprint*, plasTeX plugin for Lean formalisation blueprints, 2020, [software repository](#).
- [4] T. Zhu, P. Monticone, S. Welleck, and J. Avigad, *LeanArchitect: Automating Blueprint Generation for Humans and AI*, in *17th International Conference on Interactive Theorem Proving*, LIPIcs 382, 2026, pp. 25:1–25:16.
- [5] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, [arXiv](#).
- [6] N. Garg, *EconCSLib: AI-Assisted Lean Formalization for Economics & Computation Research*, 2026, [arXiv](#).
- [7] A. D. Brucker and B. Wolff, *Isabelle/DOF: Design and Implementation*, in *Software Engineering and Formal Methods*, 2019, pp. 275–293.
- [8] O. C. Z. Gotel and A. C. W. Finkelstein, *An analysis of the requirements traceability problem*, Proc. First IEEE International Conference on Requirements Engineering, 1994, pp. 94–101.
- [9] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, *Hints on test data selection*, IEEE Computer 11(4), 1978, pp. 34–41.
- [10] Y. Jia and M. Harman, *An analysis and survey of the development of mutation testing*, IEEE Transactions on Software Engineering 37(5), 2011, pp. 649–678.
- [11] SCSC Assurance Case Working Group, *Goal Structuring Notation Community Standard, Version 3*, SCSC-141C, May 2021.
- [12] T. Tao, *Mathematics in the age of AI*, preprint, 2026, [arXiv](#).
- [13] Lean community, *Contributing to mathlib*, [contributor guide](#), accessed August 2026.
- [14] K. Yang et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, NeurIPS 2023.
- [15] J. Storrs et al., *Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving, High Level Reasoning, and Data Extraction in Lean 4*, 2024, [arXiv](#).
- [16] M. Kripner and M. Straka, *OpenProver: Agentic and Interactive Theorem Proving with Lean 4*, 2026, [arXiv](#).
- [17] C. E. Brown, C. Kaliszyk, and J. Urban, *Agent Hunt: Bounty Based Collaborative Autoformalization With LLM Agents*, 2026, [arXiv](#).
- [18] Y. Zhang et al., *DreamProver: Evolving Transferable Lemma Libraries via a Wake–Sleep Theorem-Proving Agent*, 2026, [arXiv](#).
- [19] I. Takahara and T. Mizoguchi, *Toward Auditable AI Scientists: A Hypothesis Evolution Protocol for LLM Agents*, 2026, [arXiv](#).
- [20] D. Pratt, *Symposium: Trust via Auditable Records for Communities of AI Scientist Agents*, 2026, [arXiv](#).
- [21] A. Xin et al., *EurekAgent: Agent Environment Engineering is All You Need for Autonomous Scientific Discovery*, 2026, [arXiv](#).